

Omni-Flow: A Unified Workflow Orchestration and Distributed KV Cache Sharing Framework for Multimodal Inference

Bin Xiao^{*1} Jingfu Dong^{*2} Changran Wang¹ Yitian Chen¹
Xiaoyu Zhao¹ Yuqi Peng¹ Jianping Lin¹ Yuchen Xie¹

¹Meituan ²Peking University ^{*}Equal contribution

Code: <https://github.com/meituan-longcat/omni-flow.git>

Abstract

随着大模型 (LLM) 推理从纯文本范式演进至多模态范式, 推理系统面临三大挑战: (1) 多模态工作流的灵活编排, 其中异构计算单元表现出复杂的依赖关系与并发控制; (2) 跨进程和结点的大规模中间数据高效传输, 张量在异构角色间高速流动; (3) 跨角色高效共享 KV 缓存与模型权重, 以消除冗余的 GPU 内存占用。现有方案独立部署 LLM 与扩散模型, 缺乏对多模态流水线的系统级抽象, 导致编排逻辑分散、传输路径与特定模型强耦合, 并带来高昂的新模型集成成本。为应对这些挑战, 我们提出 Omni-Flow, 一种通过三层抽象实现多模态推理的分布式调度框架。控制流层通过 Python DSL 定义工作流, 将异构单元编排为统一的数据流图, 支持静态有向无环图 (DAG) 与动态路由, 并内置服务发现及多样化的负载均衡策略。数据流层提供超越预填充/解码分离的分布式 KV 缓存抽象, 统一内存分配机制, 并通过零拷贝、低延迟通道, 在三级分页存储层次 (GPU/CPU/SSD) 上实现跨角色直接传输。计算流层支持多轮对话中复杂的多模态前缀匹配以复用 KV 缓存, 并通过统一的 SGLang 接口接管 KV 缓存与采样逻辑, 使扩散模型可直接复用 LLM 前向路径, 遵循统一的并行语义。我们证明, Omni-Flow 通过一致的编程模型支持多种异构场景, 包括全模态对话 (LongCat-Next) 与复杂图像生成流水线 (HunyuanImage-3)。

1 引言

多模态大语言模型 (MLLM) 的应用正变得越来越广泛。然而, 多模态推理框架并未跟上这一快速演化步伐。当前的 MLLM 在推理过程中需要多个异构计算模块协同工作, 包括文本编码器、视觉编码器 (Dosovitskiy et al., 2021; Radford et al., 2021; Zhai et al., 2023)、音频编码器 (Radford et al., 2022; Du et al., 2024)、用于文本生成或扩散 Transformer (DiT) 前向计算的大语言模型 (LLMs) (Vaswani et al., 2017; Touvron et al., 2023)、扩散降噪模块 (Ho et al., 2020; Rombach et al., 2022; Peebles & Xie, 2023),

以及变分自编码器 (VAE) 编码器和解码器 (Kingma & Welling, 2014)。多模态模型架构仍在迅速演化, 不同任务表现出显著不同的模型拓扑结构。图像理解 (Liu et al., 2023; Chen et al., 2023b)、图像生成 (Podell et al., 2023; Black Forest Labs, 2024)、语音识别和语音对话采用不同的编码器与解码器结构以及不同的执行调度。因此, 在现阶段, 构建一个能够高效容纳多种模型架构的统一系统框架, 比急于优化单一固定架构更为紧迫。

如图 1 所示, 以 BAGEL (Deng et al., 2025) 和 HunyuanImage-3 (Tencent Hunyuan Team, 2025) 为代表的近期模型设计正日益在共享的 Transformer 骨干网络 (Xie et al., 2024; Wu et al., 2024;

Correspondence to: Bin Xiao
<xiaobin14@meituan.com>.

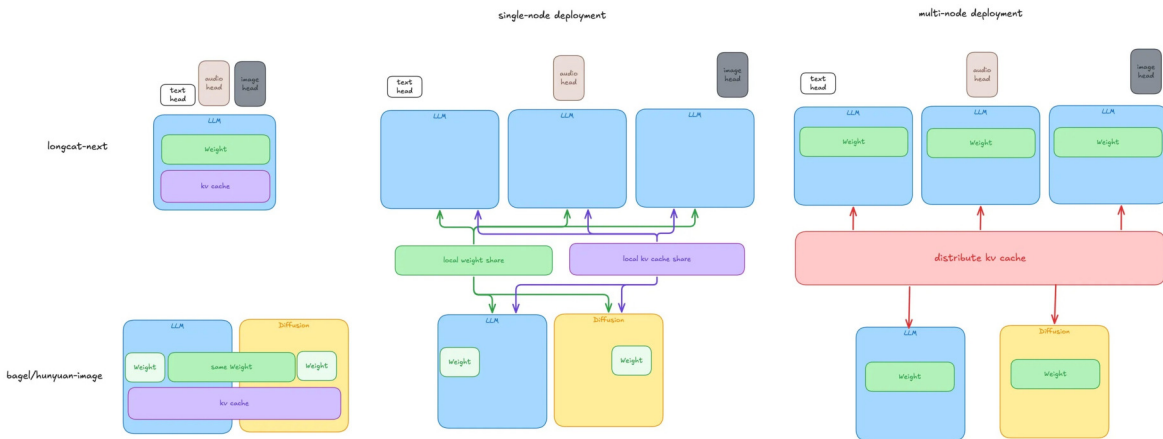


Figure 1. 多模态模型架构与收敛趋势

Emu3 Team, BAAI, 2024) 中统一理解与生成。大语言模型 (LLM) 执行跨模态理解，而扩散过程在生成阶段复用相同的模型权重，并共享 KV 缓存以避免冗余的 GPU 内存消耗。相比之下，LongCat-Next (Meituan LongCat Team et al., 2026) 采用由大语言模型和多个多模态头部组成的架构，其中文本、图像和音频头部表现出显著不同的计算负载。

任意到任意的多模态模型将多个异构模型整合为单一推理流水线。因此，中间张量和键值 (KV) 缓存不再局限于单一推理引擎，而必须在不同计算角色间高效共享和传输。这一转变引入了若干新的系统需求。资源受限的单节点部署需要本地共享模型权重和 KV 缓存，以避免 GPU 内存消耗翻倍。跨多台机器的高性能部署则需要统一的分布式 KV 缓存管理机制。同时，持续演进的多模态模型拓扑结构要求具备灵活的工作流抽象，能够容纳多样化的异构执行流水线。然而，现有的生产级推理框架缺乏一种系统级抽象，无法共同满足这些需求。

为应对这些挑战，我们提出 OMNI-FLOW，一个面向异构多模态推理的模型无关服务框架。OMNI-FLOW 围绕三个协同工作的抽象构建统一的多模态推理系统：控制流、数据流和计算流（整体架构如图 2 所示）。(1) **控制流** 提供多模态工作流及其执行逻辑的统一表示。它将多模态推理建模为支持循环的声明式有向图，其中结点代表异构计算角色，边代表数据传输通道。该抽象支持高级工作流语义，包括

或-与聚合、多产出流式传输、钻石结构、串行及多流合并。同时，它集成服务发现与多种负载均衡策略，使工作流拓扑与模型实现解耦。(2) **数据流** 通过统一的数据平面提供对中间张量和分布式 KV 缓存的统一管理，支持跨进程、跨机器和跨存储层级的数据共享与传输。针对 KV 缓存管理，OMNI-FLOW 将直接预填充到解码 (PD) 传输 (Patel et al., 2024; Zhong et al., 2024)、分页管理 (Kwon et al., 2023) 以及分层存储 (Gao et al., 2024; Qin et al., 2024) 集成至分布式 KV 缓存管理器中。该设计将推理引擎限制在仅执行模型前向计算，而将缓存分配、迁移、共享与回收等操作交由数据平面在 L1/L2/L3 存储层次间完成。此外，系统支持跨计算角色、存储层级与机器之间的直接数据共享与传输。系统还提供零拷贝、低延迟通信通道，并具备分层回退机制，同时支持同一机器上多个进程之间模型权重、KV 缓存及输入/输出参数的零拷贝共享。(3) **计算流** 为异构模型提供统一的执行接口。它支持复杂的多模态前缀匹配——涵盖完整匹配与增量匹配两种模式——以实现在涉及异构模态输出的多轮对话中复用 KV。它采用 SGLang 作为深度可定制的大语言模型推理内核，并通过统一接口掌控 KV 缓存池与采样后处理。对于兼容的扩散模型，DiT 模块可直接复用大语言模型推理引擎的执行路径与分页注意力 (PagedAttention) 基础设施 (Kwon et al., 2023)。该设计使得模型权重与 KV 缓存在统一并行执行语义下得以共享，减少重复实现工作量，并提升异构模

型间的资源复用效率。这三个抽象彼此解耦但协同工作，允许模型拓扑、数据移动与潜在计算独立演进。三者共同构成一个统一、可扩展且高效的推理服务框架，以适应快速发展的多模态模型需求。

总结而言，我们作出了以下贡献。

- 我们提出了一种统一的声明式工作流抽象，将多模态推理建模为执行图，支持异构计算角色、条件执行、多产出流、灵活的连接模式以及带有环的迭代工作流。
- 我们提出了一种统一的数据平面，用于管理中间张量和 KV 缓存的生命周期，将缓存管理、数据传输、分层存储以及零拷贝共享统一到一个框架中，以实现跨角色和跨机器资源的高效共享。
- 我们提出了一种统一的异构计算抽象，使兼容的扩散 Transformer 能够复用大语言模型推理引擎的执行基础设施，在降低模型适配开销的同时提升资源利用率。
- 我们在 SGLang 之上实现了 Omni-Flow，并在典型的多模态模型上验证了其有效性、通用性和可扩展性。我们还在附录 C 中分享了我们的 AI 辅助开发实践和工程经验。

2 相关工作

2.1 多模态推理服务与工作流编排

现代大语言模型服务系统，如 vLLM (Kwon et al., 2023) 和 SGLang (Zheng et al., 2024)，集成了成熟的一系列最优化技术，包括分页 KV 缓存管理 (Kwon et al., 2023)、前缀缓存 (Zheng et al., 2024)、连续批量处理 (Yu et al., 2022; Anyscale, 2023)、预填充-解码分离 (Patel et al., 2024; Zhong et al., 2024; Agrawal et al., 2024)、层级 KV 缓存存储 (Gao et al., 2024; Qin et al., 2024)，以及长上下文场景下的 KV 缓存压缩 (Liu et al., 2024; Yang et al., 2024; Lee et al., 2024)。这些技术显著提升了自回归推理的吞吐量和资源利用率，但主要围绕单个大语言模型的执行而设计，因此难以泛化到涉

及多个异构模型的任意到任意多模态推理。近期系统如 vLLM-Omni (vLLM Team, 2026) 和 SGLang-Omni (SGLang Team, 2025) 通过将工作流分解为多个阶段，扩展了大语言模型服务以支持多模态任务。然而，它们的执行拓扑结构仍与模型实现紧密耦合：vLLM-Omni 仅提供粗粒度阶段，导致视觉和音频编码器等模态特定组件被封装在各个模型内部；SGLang-Omni 支持更细粒度的阶段，但将拓扑规范分散在配置、路由和执行器组件中，增加了扩展和维护的复杂性。此外，这两个系统对动态控制流的支持有限：vLLM-Omni 无法按请求粒度跳过阶段，而 SGLang-Omni 仅支持粗粒度路径路由。同时，它们的流式接口依赖于潜在通信机制，且执行模型受限于有向非循环图，无法支持包含环的迭代工作流。通用编排框架如 LangGraph (LangChain Inc., 2024) 和 LangChain (Chase, 2022) 提供了显式的图抽象，但其目标是应用层智能体编排，而非多模态推理引擎内部所需的张量级调度。这些系统的代表性工作流定义见附录 B，表 1 总结了各项能力的对比。

2.2 多模态推理中的 KV 缓存管理

统一的多模态理解与生成模型（如 BAGEL (Deng et al., 2025)、HunyuanImage-3 (Tencent Hunyuan Team, 2025)、Show-o (Xie et al., 2024) 和 Janus (Wu et al., 2024)）的出现，使得跨模型 KV 缓存共享在高效多模态推理中变得日益重要。然而，现有的多模态服务系统并未提供通用的共享机制。vLLM-Omni 为 BAGEL 和 HunyuanImage-3 等模型提供了专用支持，而 SGLang-Omni 的公开版本目前尚不支持此类模型。现有方法具有模型特异性：通常在每个模型内部维护私有的张量缓存，并通过手动序列化和逐层注入的方式在自回归模型与扩散模块之间传递 KV 缓存状态。将 KV 缓存限制于单个请求和进程，阻碍了跨节点的高效共享。缺乏跨请求和交互轮次的重用，进一步导致每次推理请求都需要重新计算公共前缀。此外，由于缓存传输逻辑与模型架构紧密耦合，支持新的多模态模型需要专门的 KV 缓存迁移机制。这些局限性促使我们提

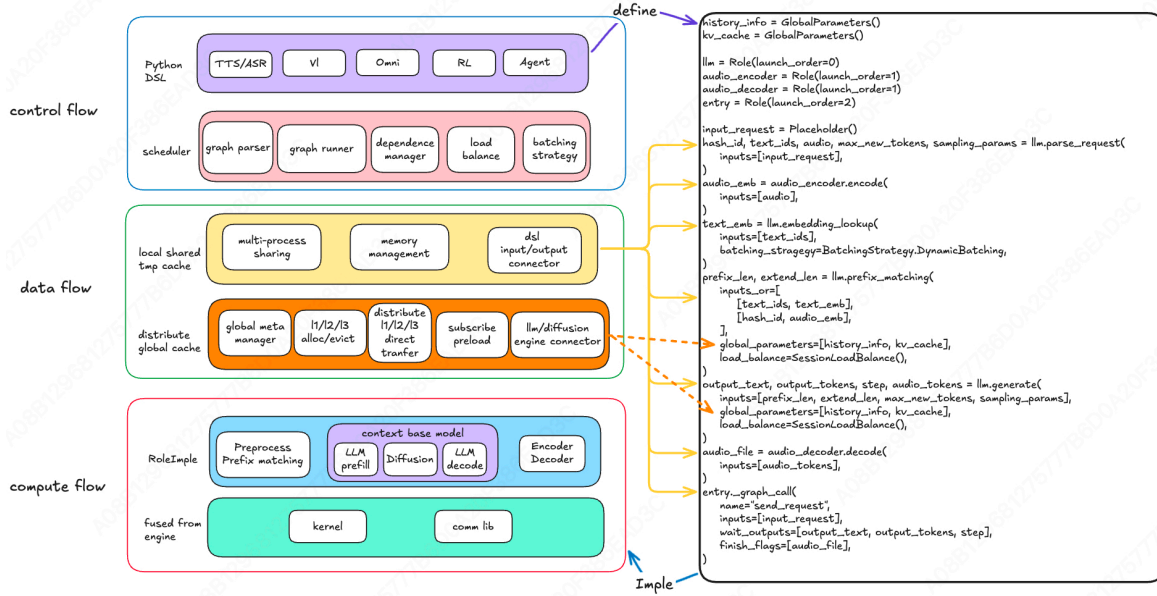


Figure 2. Omni-Flow 整体架构：控制流、数据流与计算流

出一种模型无关的 KV 缓存抽象，以支持跨模型、请求和推理轮次的分布式传输与重用。

3 控制流

控制流层将“多模态、多异构计算单元推理流水线”抽象为一种声明式图 DSL，由框架层接管所有调度与数据传输工作。这彻底消除了业务侧计算逻辑中对调度细节的负担。该设计追求两个关键目标：通用性——一套统一的抽象能够覆盖任意异构角色的拼接，例如大语言模型 (Touvron et al., 2023; Qwen Team, 2024)、扩散模型 (Rombach et al., 2022; Black Forest Labs, 2024)、视觉 (Dosovitskiy et al., 2021; Radford et al., 2021) 和音频 (Radford et al., 2022); 以及性能——完全异步且去中心化的执行流程，以在实时系统中最大化吞吐量。

首先，以该典型多模态推理场景中的控制流示例图 (图 3) 为例，我们介绍框架层面的两种核心能力以及三类依赖问题，随后针对每一类依赖问题提出相应的解决方案。

该图的执行流程如下：请求从 entry 进入，parse_request 按帧逐帧输出数据流，每帧独立激活下游阶段，无需等待所有帧完成。当帧 T_0 产生文

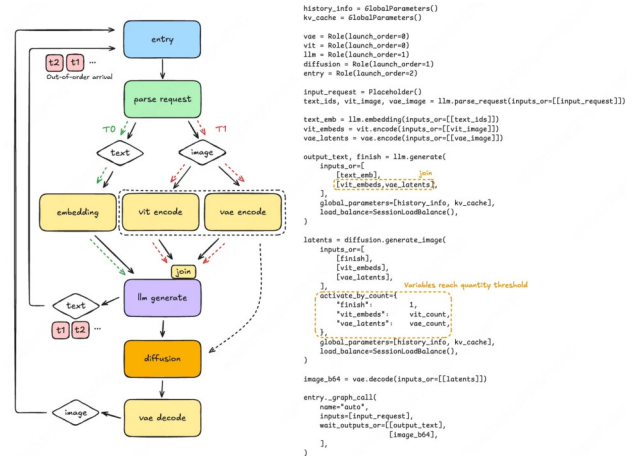


Figure 3. 控制流示例图（左：执行流程；右：DSL 定义）

本相关数据时，会立即通过 embedding 进行嵌入编码，并一次性触发 llm.generate。当帧 T_1 产生图像相关数据时，会同时触发两条编码路径 vit_encode 和 vae_encode；两条路径的结果合并后再次激活 llm.generate。完成后，llm.generate 按帧输出 token 流，一方面实时回传至 entry 供用户获取，另一方面在所有 token 生成完毕后触发扩散生成图像。图像随后由 vae_decode 解码，并返回至 entry。

上述过程体现了两项核心能力：(1) 流式逐帧激活—

Table 1. 开源通用推理框架的能力对比。

Capability	vllm-omni	sglang-omni	Omni-Flow
Dynamic Branching	✗	Δ (route_fn)	✓
Streaming Output	Δ (async_chunk)	Δ (stream_to)	✓
Loop / Cycle	✗	✗	✓

`parse_request` 在每个帧产生后立即独立激活下游阶段，无需等待所有帧完成；(2) **运行时动态分支选择**——路由根据每帧的输出类型自动进行（帧 T_0 采用嵌入路径，而帧 T_1 同时触发 `vit_encode` 和 `vae_encode`），这也为框架支持循环图结构奠定了基础。

这两个核心能力由静态图和动态图支持并完成，具体详见第 3.3 和 3.4 节。

同时，实施上述过程给该框架带来了三个核心调度挑战：

问题 1：单源数据的多路径收敛——下游同步问题。在帧 T_1 中，`parse_request` 生成图像相关数据，随后同时经过两条独立的编码路径：`vit_encode` 和 `vae_encode`。最终，两个结果必须汇聚到 `llm.generate` 以触发其执行。由于 `vit_encode` 和 `vae_encode` 处理数据的速度不同，无法保证两者在汇聚结点同时到达。因此，`llm.generate` 必须等待两条路径均准备就绪后才能被激活，而不能在任一路径到达时立即触发。

问题 2：有序流式输出但下游可能存在无序到达——到达顺序保证问题。`llm.generate` 流式输出 token，每个文本帧在逻辑上严格有序。然而，图中从 `text` 到 `entry` 的路径是一条返回边。在这些帧经过网络传输和异步调度后，它们在 `entry` 处的到达顺序可能与其生成顺序不一致——例如， t_2 可能先于 t_1 到达。如果 `entry` 将这些无序到达的 token 流式回传给用户，用户接收到的文本序列将偏离生成顺序。

问题 3：路径数量仅在运行时确定——动态同步问题。为提高吞吐量，该框架支持分段多帧生成，但反过来引入了动态同步问题：

`diffusion.generate_image` 必须同步文本嵌入、条件图像编码（来自 `vit_encode` 和 `vae_encode`）以及扩散参数——这些均不可缺失。然而，每条请求的条件图像路径数量各不相同：在文本到图像 (t2i) 生成中为 0，在文本与图像混合 (ti2i) 生成中为 N 。框架无法在解析阶段确定 N ，但在运行时必须正确实现“在触发前同步所有 N 路径”，以避免提前触发或无限等待。

这三个问题可以归类为三种不同的依赖关系：问题 1 是钻石型依赖（单源数据分流后在下游汇聚），问题 2 是序列依赖（按顺序生成的多帧数据可能在下游无序到达），问题 3 是多流合并（单源数据路径数量动态变化，需等待所有数据到达后才能激活）。针对这三类问题的解决方案详见第 3.2 节。

Omni-Flow 将调度复杂性完全封装在框架层：用户通过 Python DSL 声明字段级数据依赖关系，框架在解析阶段将其编译为静态执行计划。运行时，该计划由去中心化的结点调度器通过表查找驱动——使业务侧的计算逻辑完全摆脱调度细节的负担。

3.1 图的定义

用户通过 Python DSL 声明图结构，该 DSL 包含四个核心构建块：

- **角色：**计算角色（例如，LLM、Diffusion）。`_interface()` 声明输入/输出字段和调度策略，而 `_graph_call()` 声明子图调用入口点和完成条件。
- **占位符：**在图的入口处注入的外部数据字段。
- **全局参数：**会话粒度的全局参数（例如，键值缓存），在各个角色间共享，其生命周期由框架管理。

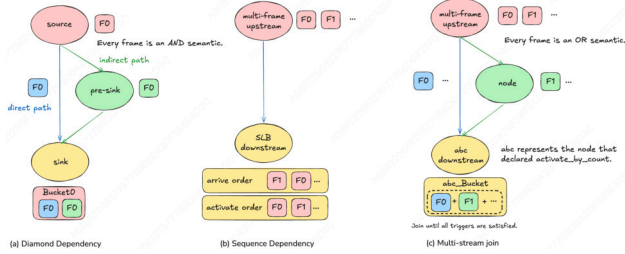


Figure 4. 三种依赖关系：钻石型、序列型和多流连接

- **RoleRef / GlobalParametersRef:** 跨图引用，支持单个角色实例托管多个图。

或-与输入语义。每个 `inputs_or` 通过一个二维列表表示：外层列表代表“或”组（只要任意一组就绪即触发），内层列表代表“与”组（仅当组内所有字段均到达时才触发）。该结构在推理中建模分支逻辑，无需编写任何显式的条件判断代码——框架会自动根据“哪个或组最先完全满足”来选择执行分支。

3.2 三种依赖关系的实现

这三种依赖关系（如图 4 所示）在现实世界的流水线中经常同时出现且相互重叠（参见附录 A）。如果为每种情况单独手动编写调度逻辑，代码将变得极其脆弱且难以复用。

为了解决这三类依赖关系，Omni-Flow 抽象出一个 **Bucket** 模型。桶的定义如下：

桶的定义：桶是用于收敛点的容器，在触发当前节点的前向传播之前，用于同步多路径输入。如果某个节点的某个或组具有 ≥ 2 个直接前驱（即多条路径在此汇聚，满足钻石依赖关系，称为收敛点），则框架会为其分配一个桶；反之，直接连接的节点（线性链中的中间节点、源节点或单前驱节点）不持有桶，每当上游帧到达时立即触发前向传播，无需等待同步。也就是说，只有在需要多路径汇聚的位置才存在桶。

桶的身份：四元组。每个桶由一个四元组唯一标识 (`gc_rid`, `owner_node`, `or_group`, `activation_id`)：

- `gc_rid` — 关联的 `graph_call` 调用，用于跨请求隔离；
- `owner_node` — 标识桶所属的结点；
- `or_group` — 拥有结点的或组（同一结点的不同或组持有独立的桶）；
- `activation_id` — 区分属于同一结点的同一 OR 组但对应不同帧的桶。

四重正交化每个可能产生数据不匹配的并发轴；任何维度的缺失都可能导致跨桶数据泄露（桶冲突）。虽然前三个维度可在解析阶段确定，但 `activation_id` 会在运行时递归计算，以确保前一帧与后一帧落入不同的桶中，而同一源和同一帧的数据则落入确切相同的桶中。

桶的内部状态与触发条件。在解析阶段，到达桶的每条路径被称为一个静态槽。桶维护四个集合：`required`（必须同步的完整槽集合）、`received`（已接收数据的槽集合）、`slot_eos`（已接收流结束指示的槽集合），以及 `unreachable`（由上游报告为不可达的槽集合）。桶的触发准则为：

$$required \subseteq received_satisfied \cup unreachable$$

当所有必需路径都已到达或被确认为不可达时，桶就会触发并安排当前节点的前向传播。

桶生命周期：桶无需预先注册——框架仅在上游节点从特定路径发送帧后，才惰性地创建相应的桶；一旦桶被触发或确定不可达，即可立即回收。

3.2.1 钻石依赖

术语：菱形结构包含三种类型的结点——分叉源结点 (`source`)（例如，`parse_request`）、中间结点 (`pre-sink`)（例如，`vit.encode`）以及收敛点结点 (`sink`)（例如，`llm.generate`）。

核心机制：使用分叉源的身份和帧序列号作为 `aid` 的种子。该框架通过哈希一组特定的身份信息，为流向收敛点的每条消息计算一个 `aid`。为了确保来自同一源的多条路径落入同一个桶中，关键在于种子的选择：所有路径统一使用其共同的分叉源身份作为种子，而非各自直接上游结点的身份。因此，无

论路径经过哪些中间结点或路径长短如何，代入哈希函数的种子完全相同，从而生成一致的 `aid`，使得消息被置于同一桶中；反之，后续帧对应不同的分叉源帧序列号，导致种子不同，自然落入新的桶中。

3.2.2 序列依赖

该框架将顺序保持功能集中在 `SessionLoadBalance` (SLB) 结点上，而所有其他结点完全不受排队影响。对于每个排队键 $q = (session, gc, interface)$ ，保持顺序的结点维护一个期望的序列号 `expected`，并强制执行两个不变：(i) 它仅允许活性值的序列号与 `expected` 恰好匹配的请求通过；(ii) 仅在当前序列号对应的 OR 组已被确切确定后，才推进 `expected`。

这种收敛性成立的原因在于，任何具有会话内状态依赖关系的结点都会被明确声明为 SLB 结点，而无状态结点（如轮询或广播）则不需要保持顺序。通过在离开 SLB 结点时重新分配序号，将保持顺序的信息逐段传递，使得中间的普通结点可以完全无序地执行。这种方法避免了引入多余的序列化点，同时保证了端到端的顺序。

序列号来源：`seq` 和 `first_seq` 标签嵌入在消息中。Algorithm 1 中的 `seq` 表示上游结点写入的标签（即 `SeqId`），并随数据帧逐跳传播。

3.2.3 多流连接

对于此类结点，声明了 `activate_by_count`。当有人入边到达时，框架会强制将这些信号的桶标识重写为同一常量，导致多条路径汇聚到同一个桶中。该机制分两个步骤处理：

步骤 1：合并桶。在默认情况下，不同的 OR 组以及同一结点的不同活性值落入各自独立的桶中。由于 `activate_by_count` 结点的语义规定“单次活性值从所有上游路径收集字段”，因此框架在输入到达时强制将所有输入分配到确切的同一个桶中，从而在结点粒度上实现多路径输入的均匀累积。

步骤 2：通过字段级阈值触发。合并后的桶独立计

Algorithm 1 Sequence-Order Advancement per queue key q

State per q : exp (expected seq), $pending[seq]$ (ready groups), $settled[seq]$ (done groups), $busy$

on GROUPREADY($q, seq, g, first$):

if first time: $exp \leftarrow first$

$pending[seq].add(g)$; TRYADVANCE(q)

proc TRYADVANCE(q):

while $\neg busy$ and $pending[exp] \neq \emptyset$ **do**

$busy \leftarrow true$; release $pending[exp]$ for execution

end while

on GROUPSETTLED(q, seq, g): {fired or DEAD}

$settled[seq].add(g)$

if $settled[seq]$ covers all OR-groups of this node:

$exp += 1$; $busy \leftarrow false$; TRYADVANCE(q)

算每个触发字段。一旦某个字段的到达帧数达到声明的阈值，该字段即被视为满足；只有当所有必需字段均满足时，前向传播才会被触发。

3.3 图解析与静态计划

用户通过 Python DSL 编写的图定义是声明式描述，框架在启动时将其转换为运行时可直接使用的数据结构。这一转换带来了两个直接好处：首先，配置错误在任何请求到达之前即可暴露（快速失败）；其次，运行时的热点路径简化为纯粹的表查找，无需重复推导。

图解析器 (GraphParser) 将用户编写的图定义模块作为输入，通过静态反射提取每个结点的接口声明和字段级数据依赖关系，并输出一个结构化的 `Graph`。在多图场景中，它还负责基于 `gid` 的路由以及跨图一致性验证。

静态计划 (StaticPlan) 将解析后的 `Graph` 作为输入，并预先计算运行时调度所需的所有索引：

- **边表 (边)**：字段级别的有向边 {源} $\rightarrow$ {目

标}, 用于描述数据流。

- **控制边 (CtrlEdge):** 指定上游结点在完成一次产出后, 应向哪些下游结点发送 ARRIVED、SLOT_EOS 或 UNREACHABLE 信号。
- **钻石根:** 通过反向 BFS 定位每个收敛结点的共同祖先, 用于计算 `activation_id`。
- **关键槽位:** 标记具有唯一上游路径的字段槽位; 当该路径不可达时, 可直接判定该桶为 DEAD。

3.4 动态图

动态图层负责将静态计划的执行结果转换为实际的请求执行流程, 同时处理钻石同步、顺序保持和不可达性传播等多种复杂场景。其设计目标是用最少数量的正交机制覆盖所有场景, 确保每个结点的处理逻辑简单且统一。

核心调度器 (NodeController) 作为每个图结点的执行单元, 负责管理请求从入站到达至出站分发的完整生命周期:

入向路由: 它根据静态规划决定, 传入的消息是否应在收敛结点进入桶中累积, 或在直接连接的结点直接触发前向传递。

出站调度: 它根据每次执行后的 CtrlEdge 动态地将帧向下流路由——如果存在输出字段, 则向相应的下游结点发送 ARRIVED 信号以激活该结点; 如果字段不存在, 则发送 UNREACHABLE 信号以静默跳过该分支, 实现运行时动态分支选择。零值、异常和取消操作均通过 UNREACHABLE 路径透明传播。

Bucket 状态机负责在收敛结点进行数据累积并触发决策。其核心逻辑在“所有必需的上游路径均已到达或变得不可达”时触发一次前向传递。标准的连接桶根据上游源的粒度做出此判断, 而 `activate_by_count` 桶则根据字段级别的计数阈值做出判断。

SeqId 分段顺序保持机制维持了流帧的序列信息。顺序保持标签 SeqId 仅由多帧结点和 SLB 结点生成, 而普通结点仅透明地传播该标签。这使得顺序

保持逻辑集中于顺序生成结点, 而不侵入中间的各个结点。

3.5 其他机制

本节总结了几种相对独立的机制, 并概述了它们的核心概念:

不可达性传播与取消: 当一个结点遇到零产出、异常、取消或字段不匹配时, 它会向其 `ctrl_edges` 发送一个 UNREACHABLE 信号 (区分空、错误和取消); 如果命中关键槽, 则该桶被标记为 DEAD, 且此状态会递归地向下游传播; 如果命中非关键槽, 则仅记录在 `unreachable_inputs` 中, 该桶仍可正常触发。

任务完成判定 (ActivityTracker): ActivityTracker 维护一个全局的进行中结点集合; 当结点被激活时注册, 其前向传播和控制信号分发完成后注销。一旦进行中集合变为空, 即判定图计算 (GC) 已结束, 随后触发 `system_finish`。

跨图调用序列化 (GCScheduler): 同一会话内的并发 `graph_call` 实例会在共享的全局参数 (如 KV 缓存和历史记录) 上引入竞争条件。GCScheduler 强制实施会话级别的先进先出队列, 仅允许队列头部的实例执行, 并在前一个图调用完全退出后才释放下一个; 不同会话间的执行仍保持非阻塞。通过将序列化点提升至调用入口层, 该框架完全避免了因结点内队列与延迟回调交错而引发的死锁问题。

并发模型: 控制流层不使用业务级锁; 相反, 序列化通过“单线程事件环 + 无分支调用链”实现。入站消息沿连续的 `await` 链展开: `send` → `on_inbound` → 桶累积 → 触发 → 转发, 不创建分离的新任务。这保证了当上游 `send` 在进程内返回时, 下游 `forward` 已经自动完成, 从根本上消除了回调乱序引发的竞争条件。在环形拓扑下, 通过 (`origin`, `node`, `or_group`) 对 UNREACHABLE 信号进行去重, 以防止环内无限递归。

服务发现 & 负载均衡: ServiceRegistry 基于 Redis (Redis Ltd., 2009) 实现分布式注册与发现, 通过

独立连接维持心跳，以隔离 Redis 波动对主数据路径的影响。超时的实例会被剔除，缩放过程中路由会自动重建。在此注册表基础上，**LoadBalance** 提供三种策略：用于无状态分发的 RoundRobin，以及用于会话亲和性路由的 SessionLoadBalance（亲和状态在 Redis 中持久化，支持跨进程共享，同时也支撑了序列级依赖追踪）。

4 数据流

当前的多模态模型正朝着统一的骨干架构收敛 (Deng et al., 2025; Tencent Hunyuan Team, 2025; Xie et al., 2024)——其中大语言模型负责跨模态理解，而扩散模型负责生成，两者共享完全相同的 Transformer 权重和键值缓存。与此同时，可预测的超长上下文任务 (Beltagy et al., 2020; Lin et al., 2024) 将不可避免地催生对键值缓存压缩与紧凑化的需求 (Liu et al., 2024; Yang et al., 2024; Lee et al., 2024)：压缩后的键值缓存必须能够被多个下游角色重复使用，而非仅限于单一使用者。因此，键值缓存的多角色共享能力已不再是奢侈品，而是演进发展的架构必然要求。

现有的 KV 缓存共享方案（如图 5 所示）相对零散：预填充/解码分离 (Patel et al., 2024; Zhong et al., 2024; Agrawal et al., 2024) 依赖专用的点对点传输通道，本地容量扩展利用 L2/L3 存储 (Gao et al., 2024; Alizadeh et al., 2024)，跨服务共享则依赖 L3/L4 全局缓存 (Qin et al., 2024; Srivatsa et al., 2024)。这三种机制各自独立运行，难以在超过少量固定角色之外进行组合。

为解决此问题，我们提出一种以 **全局参数池** 为中心的统分布式 KV 缓存抽象（如图 6 所示）。L1/L2/L3 存储层次中的分配与回收操作均集中于单一数据平面，所有数据传输——无论是不同角色之间还是同一角色的副本之间——均通过直接的数据平面通道统一路由。与仅开放单个固定传输路径的预填充/解码分离方案不同，全局参数池构建了一个全连接的数据网络：任意角色或副本均可向其他任意角色或副本推送或拉取数据，无需任何硬编码

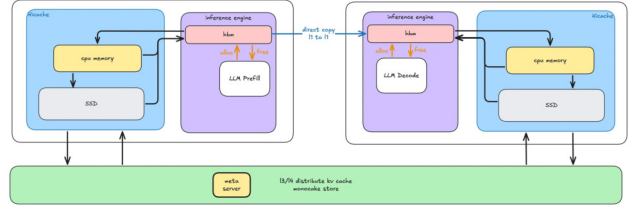


Figure 5. 现有的 KV 缓存共享方案

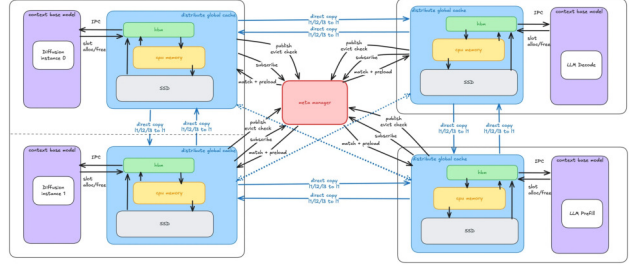


Figure 6. 统一的分布式键值缓存架构

的配对假设。推理引擎仅负责前向计算，完全忽略数据局部性和所有权。这种整合设计可自然扩展至任意数量的协作角色，且无需修改架构。

数据流子系统采用客户端-服务器架构：**MemoryManager** 作为核心服务器组件，统一管理所有内存池并处理 RPC 请求，而 **MemoryManagerClient** 则嵌入到每个 Role 进程中，通过 ZMQ (Hintjens, 2013) IPC 与服务器通信。内存根据使用情况分为三大类池：

- **全局参数池**：KV 缓存的指定目标位置，采用分页的分层存储架构 (Gao et al., 2024) (L1 GPU / L2 CPU / L3 SSD)，根据热点/冷点数据访问模式自动在各层级间迁移数据。
- **临时缓冲池**：用于中间数据的中转枢纽，嵌套在系统内部，可自动完成跨不同角色接口的输入和输出张量的零拷贝、低延迟传输。
- **权重共享**：用于模型权重的共享层，基于结构化、通用的名称进行匹配，使得同一 GPU 上的多个共置角色可以重用完全相同的物理权重，将 GPU 内存开销降低至单个副本的水平。

4.1 全局参数池

与现有的 KV 缓存方案（引擎内部管理一个基数树，维护点对点的预填充/解码直接传输通道，并独立管理本地 L2/L3 存储）不同，全局参数池的核心发散之处在于其全面接管了管理权：

- 所有高带宽内存（HBM）上的键值缓存槽位分配与回收均由数据平面统一管理。推理引擎仅负责前向计算，对数据的位置和所有权完全不知情。
- L1/L2/L3 三级存储层次结构完全注册在分布式元数据中心内。与采用点对点预填充/解码直接传输不同，共享通道被变换为订阅与预加载机制——数据发布时会自动推送通知，订阅者按需拉取数据。
- KV 缓存分片的跨副本检索路径由全局元数据中心根据布局信息和候选组合算法统一确定。分片可从任意源结点获取，与该结点的具体角色或副本索引无关。

子系统按四层组织：物理分页层 →，多级时隙引擎 →，分布式元数据协调层 →，完整数据流闭环。

4.1.1 物理分页层

物理分页层负责在三个内存层级（L1 GPU / L2 CPU / L3 SSD）之间进行底层存储管理，并为槽引擎在迁移过程中提供跨层级复制原语。它不强制采用单一的张量布局，而是支持灵活的组合式定义：每个全局参数由两层元数据规范描述——外层 `sub_name` 维度（例如，每层 Transformer 对应一个条目），以及内层 `slice_name` 维度（例如，每头的 KV 缓存 (Su et al., 2021) 缓冲区，或任何未来的切片类型），从而无需修改任何分配代码即可容纳异构模型架构。上层通过提供三个坐标 (`sub_names_tuple`, `page_idx`, `slice_idx_tuple`) 来获取任意张量视图，其笛卡尔积可自动覆盖所有层 × 页面 × 切片组合。

4.1.2 多层滚筒引擎

Slot Engine 将页面映射到三层物理介质上。驱逐策略由一个 TinyLFU (Einziger & Friedman, 2014) Count-Min Sketch 驱动，该算法通过周期性衰减估算每页的访问频率，并为一个四层频率感知的 LRU（冷 / 温 1 / 温 2 / 热）提供依据，始终从最冷的块中回收资源。不同于整数引用计数器，每个槽位维护四个 `Set[str]` 跟踪器（活跃请求、写任务、驱逐任务和读回任务），通过集合的幂等性保证驱逐的安全性，彻底消除负计数和内存泄漏问题。跨层级迁移使用自定义 CUDA 内核，实现 GPU 优化的 NHD 块栈布局与 CPU/SSD 使用的扁平序列化布局之间的零拷贝格式转换。最后，对同一页面的并发未命中通过飞行中的上传字典进行去重：首个请求者作为所有者执行物理 I/O，后续请求者等待并复用结果——将 N 次并行上传压缩为单次操作，防止“雷鸣羊群”效应放大。

4.1.3 分布式元数据协调

以 Redis (Redis Ltd., 2009) 作为唯一可信来源，所有写操作通过 Lua 脚本原子执行，读操作则通过流水线批量处理，以最小化往返时间 (RTT)。协调层围绕四个功能块组织。

布局识别与跨结点一致性。每个结点将其物理布局元数据编码为一个 JSON 文档，并取其 SHA-256 哈希值的前 16 个十六进制字符作为全局唯一的布局 ID。当单个结点仅覆盖页面的一部分（例如，结点 A 持有 K，结点 B 持有 V）时，候选组合算法会枚举所有能覆盖完整页面的结点-布局并集，按组合大小、出现频率，然后按字典序排序。

会话前缀匹配。给定一个会话 ID 和查询页面序列，系统在 Redis 中缓存的页面链上执行最长前缀匹配，自动截断过期的尾部，并逐页验证候选组合的副本可用性——在遇到第一个无有效提供者的页面时停止。

副本发布与提供者选择。完成计算后，结点会注册为相应页面的副本提供者。在查询过程中，会选择优先级最高的候选组合提供者，优先选择覆盖更多

页面的结点，以减少跨机器传输开销。

三层保护下的原子性逐出。所有逐出检查均在单个 Lua 脚本中执行，以消除 TOCTOU 竞态：(1) 复制体引用锁—目标页面是否存在来自其他结点的活跃锁；(2) 会话运行锁—是否有关联于此页面的正在进行的推理请求；(3) 候选组合完整性—移除后每个组合是否仍至少保留一个完整的复制体。该脚本确认以上三个条件，并原子性地执行删除操作，确保检查与动作之间不存在间隙窗口。

4.1.4 完整数据流环

初始化。当上层创建全局参数时，API 分配物理分页张量，将内存区域注册到 RDMA (Gangidi et al., 2024) 引擎，建立 L1/L2/L3 插槽引擎结构，并将布局及其候选组合注册到 Redis。

推理写 → 发布 → 预加载。完成计算后，请求调用发布接口，将页 ID 映射到本地槽位，注册该结点为副本提供者，并将新页追加到会话的页链中。下游预加载会自动触发，通过批量通知传递来减少冗余的跨结点消息。

通过 RDMA 实现跨结点需求拉动。在接收到预加载通知或本地缓存未命中时，订阅者首先在其本地多级存储中进行搜索。若未命中，则查询 Redis 以获取远程提供者，并启动封装在严格“搜索-传输-释放”生命周期中的 RDMA 批量读取：在本地槽位上注册搜索标记，执行跨结点直接读取，并在完成时释放标记。

写时复制与碎片化页面处理。当增量数据被迫加到由多个活跃请求共享的页面（引用计数 > 1 ）时，会自动触发写时复制。对于未填满完整页面的部分写入，脏键跟踪机制会记录哪些段已被覆盖，从而确保后续读取返回正确拼接的张量视图。在多模态流水线中，碎片化页面始终以完整形式保留，因为碎片化页面可能跨越不同模型角色生成的内容（例如，由大语言模型组件写入的键值缓存，扩散组件无法重新计算），因此部分淘汰或截断是不安全的。

内存压力下的淘汰。当本地 HBM 处于压力状态时，插槽引擎会识别出低优先级的候选对象，并向 Redis

提交一个三层保护的淘汰请求。淘汰决策从不会阻塞活跃的推理任务：§4.1.2 中描述四个 `Set[str]` 引用追踪器提供了淘汰与推理任务之间的自动互斥。

4.2 临时缓冲池

在工作者之间传输的中间张量（活性值、编码器输出等）通过预先分配的板块进行管理——该板块是一个连续的、字节可寻址的张量，划分为固定大小的块，按需提取，从而避免了每次请求都进行操作系统级别的内存分配。

引用计数 + LRU 双轨复用。每个已分配的块均维护一个引用计数：在分配时初始化为 1；当下游消费者通过零拷贝 `add_ref` 重用该块时，引用计数加 1；仅当 `free` 将计数减至 0 时才进行物理回收。同时，嵌入了一个基于 `OrderedDict` 构建的 LRU 缓存——根据 `cache_key` 借用已有写入的块；缓存命中时，`ref` 加 1，但块仍保留在缓存中不被弹出，允许多个并发请求借用同一份只读数据。在内存不足 (OOM) 事件中，引擎仅淘汰引用计数为 0 的空闲缓存块，而活跃块则通过引用计数得到保护。此外，`find_alloc_by_ptr` 支持通过任意字节偏移进行 $O(\log N)$ 复杂度的反向查找，追溯到其父分配块，解决外部获取的张量切片指针位于块中间而非起始地址时的溯源问题。

与传统方案的一个关键区别在于，跨主机传输采用按需拉取 (pull-on-demand) 模式，而非推送 (push) 模式——数据生产者从不主动推送数据；相反，只有当消费者需要时，才会通过 RDMA 进行拉取，从而完全消除冗余传输，并避免了管理接收端缓冲区的开销。

4.3 权重与 KV 缓存共享

模型权共享。通过修改 PyTorch 的工厂函数 (`torch.empty`、`torch.zeros` 等)，在模型构建期间所有参数分配均重定向至 meta 设备，从而在构建时实现零 GPU 内存占用。构建完成后，框架根据模型架构以及 TP/EP 拓扑生成名称键，分配或

复用跨进程 IPC 共享张量，由拥有者进程将检查点数据写入共享 GPU 内存，跟随者进程则使用一个 `SkipCopyParameter`，其 `copy_()` 为无操作——由于所有进程物理上指向同一内存块，冗余拷贝会无声地破坏彼此数据。共享权重初始化采用客户端轮询而非服务端条件变量：若所有跟随者阻塞于 `condition.wait()`，将导致 `MemoryManager` 线程池饱和并死锁；客户端每秒轮询一次即可完全避免此问题。总体效果是，多角色部署仅消耗单个副本的权重内存。

KV 缓存共享。多个共置的角色（例如，大语言模型和扩散模型）共享一个统一的 KV 缓存池，其大小设置为所有角色提议中的极小点，防止任何单一角色过度分配。一个 `kv_pool_barrier` 解决了一个微妙的时序危险：较早完成权重加载的角色会观察到人为偏高的空闲内存，并因此高估可用的 KV 池大小；该屏障确保所有共置角色在评估池大小前均已完成权重加载。

5 计算流 × SGLang

原生多模态模型 (NMM) 推理框架的计算平面解决了根本性矛盾：模型拓扑在不同模态间高度灵活（图像理解、生成、语音转录、语音对话各自具有不同的编码器/解码器），但推理执行必须保持高度高效。策略是将所有模态统一为单一的 `HiddenState` 表示，再进入大语言模型 (LLM) 主干，从而使得经过验证的 LLM 优化技术——如 KV 缓存复用 (Kwon et al., 2023; Gao et al., 2024)、共享内存输入输出 (Dao et al., 2022; Dao, 2023)、推测解码 (Chen et al., 2023a; Cai et al., 2024) 以及权重量化 (Frantar et al., 2023; Lin et al., 2023)——能够均匀应用于任意模态。`compute_flow` 层将 SGLang (Zheng et al., 2024) 视为一个深度可定制的 LLM 内核，通过统一接口接管其内存与采样，并强制模态编码器和扩散模型收敛到相同的前向执行路径。以下是三个核心机制的描述：前缀匹配、接口抽象与 KV 接管，以及扩散模型复用 LLM 路径。

5.1 前缀匹配

传统的 LLM 前缀缓存基于 token ID 序列，这在多模态推理中是不足的，因为生成的图像、音频、视频等输出无法在后续轮次中简单地重新分词。该框架通过将前缀匹配分解为两种共享潜在机制的模式来解决这一问题。

完整匹配适用于没有复杂模态输出的场景。服务器运行所有模态编码器以获得确切的片段长度，将得到的隐状态连结成一个完整序列，并进行逐页前缀匹配。为避免冗余编码，每个编码器维护一个基于原始输入负载（图像 URL、base64 等）哈希值的跨请求 LRU 缓存，因此缓存命中时可完全跳过编码器前向传播。

增量匹配在输入引用了先前模态输出但无法通过重新解析恢复其 token ID 时适用。客户端携带一个 `step` 偏移量，声明其已获取的数据位置；服务器将历史记录截断至该位置，并仅处理新的增量数据。若 `step` 为负值或缺失，则退化为追加语义，消耗当前全部历史记录。

两种模式共享相同的潜在**链式分页哈希**机制：每个页面的哈希编码了其前驱页面的哈希，因此只有当整个前缀逐页匹配时，哈希值才相等。这将基数树语义压缩为一个扁平的哈希字符串，可在进程和副本之间直接比较，而无需维护共享的前缀树。

LLM 的前缀复用进一步分解为两个语义层。**Layer 1—可重计算的完整输入** (`emb`，占用空间小，存储于 CPU)：来自不同模态的所有输入被统一抽象为隐状态。即使所有 GPU KV 缓存丢失，系统仍可依赖此 `emb` 执行全新的预填充并重建 KV，确保正确性和可恢复性。**Layer 2—KV 缓存匹配，“可用则使用”**：GPU 上的 KV 缓存命中完全是机会性的——系统仅复用命中部分，未匹配的部分则回退至 Layer 1 进行重计算，从而保障性能。解耦这两层意味着昂贵且易失的 KV 缓存仅以尽力而为的方式复用，而廉价可靠的 `emb` 则作为安全网。

与每次请求仅输出单个帧不同，前缀匹配阶段按需生成多个帧，以最大化异步并发性。如果存在未匹

配的历史前缀，则首先发出一个重新计算帧，以独立重建 KV 缓存。随后，每个模态段作为独立的帧输出，包含其偏移量、长度和隐状态片段。这种分段输出机制实现了流水线预填充：下游结点在接收到每个帧后即可立即开始计算该段内容，无需等待完整序列完成。结合 OR 组活性值机制，不同段的准备和计算在流水线中重叠进行，将请求内部的串行等待转化为跨段的异步并行处理。

5.2 SGLang 与 KV 接管的接口抽象

该框架并未直接使用 SGLang 的原生内存和采样逻辑，而是通过统一的插件单例剥离 SGLang 运行时，将完全控制权移交至框架的数据平面。此次重构涵盖两个主要方面。

抽象的输入/输出参数接口。每个请求都携带精确的元数据（会话 ID、前缀/后缀/预填充长度、最大新 token 数、物理槽位、页面 ID 等）。在执行时，输入张量（如缓存的隐状态）被复制到持久化的 GPU 缓冲区，并附加到前向批量；完成时，生成的 token 和隐状态片段被复制回。流入和流出的张量由可配置的键列表驱动，使得模型 I/O 成为声明式接口而非硬编码逻辑。

整体式 KV 缓存接管。SGLang 的原生基数树插入被完全绕过。前缀匹配直接返回由框架预计算的物理槽位，请求完成仅需将状态发布回数据平面。真正的 KV 缓存所有权位于框架的全局参数内存池中：槽位分配、命中/未命中划分、提交后发布，以及从业务侧槽位 ID 到引擎页 ID 的转换均由数据平面管理，而 SGLang 的注意力核函数仅负责向这些槽位写入数据。

5.3 扩散重用大模型路径

一个关键的工程决策是，扩散模型 (DiT) (Peebles & Xie, 2023; Rombach et al., 2022) 模块不使用原生的扩散运行时，而是复用完整的大型语言模型 (LLM) 基础设施——相同的引擎和调度栈、相同的分页键值缓存以及相同的共享权重机制。这种复用通过一个共享基类与运行时插件选择来实现：LLM 和 DiT

均继承自同一套键值与权重管理基类，而环境切换在运行时决定各自的具体实现。DiT 的降噪步骤被组织为一系列沿标准分页注意力路径执行的操作，图像数据和时间步数据作为前向批量中的额外字段传递。

其动机在于并行对齐。权重和 KV 缓存共享要求两个角色在 TP (Shoeybi et al., 2019)、EP (Fedus et al., 2022; Jiang et al., 2024) 及其他并行策略 (Huang et al., 2019; Zheng et al., 2022) 中采用相同的分割维度。原生扩散堆栈不支持专家并行，使得跨角色共享变得不切实际。将 DiT 移植到 LLM 路径上会迫使两个角色采用相同的并行语义，从而能够直接共享物理权重和 KV 缓存。

6 使用 Ray 部署

单个副本作为资源单元。资源配置以单个角色副本为最小粒度 (Moritz et al., 2018)——用户仅需声明每个副本的 GPU 数量和结点数量，框架将自动推断出集群的整体拓扑结构。副本是同质的，物理卡数量会自动切分，只需修改 `replicas` 参数即可在单个资源组内实现弹性缩放。

层抽象——物理卡资源复用。位于 `layer: 0` 的角色决定放置组 (PG) 拓扑并物理请求 GPU 分配；位于 `layer >= 1` 的角色声明 `num_gpus=0`，绕过 Ray 资源配额，但通过注入 `CUDA_VISIBLE_DEVICES` 复用由第 0 层已分配的物理显卡。这使得 Ray 的资源管理与实际物理分配解耦，成为多角色、共置部署中资源复用的基础。

7 结论与未来工作

Omni-Flow 通过三层抽象——控制流、数据流和计算流——为多模态推理场景提供了统一的工作流编排、数据传输以及键值缓存共享解决方案。该框架已成功支持三种典型场景：DeepSeek-V2 (DeepSeek-AI, 2024) 纯大语言模型推理 (PD, 流式 token 输出)、LongCat-Next (Meituan LongCat Team et al., 2026) 多模态对话（带有不同头部的 LLM）以及 HunyuanImage-3 (Tencent Hunyuan Team, 2025)

图像生成流水线 (LLM+DiT)。

该框架目前仍处于早期阶段，未来的工作方向包括：(1) 性能最优化是下一阶段的重点，核心目标是向实时推理演进；(2) 深化分布式 KV Cache——支持更多注意力变体 (Ainslie et al., 2023; Dao, 2023)，层级传输策略，不同 TP 配置下的高效传输，CP 支持，智能选择 KV 分片等；(3) 更复杂的调度策略，例如基于缓存感知的调度，根据每个副本的 KV Cache 命中状态做出路由决策；(4) 强化学习集成 (Ouyang et al., 2022; Kimi Team, 2025)——目前模型权重已由框架统一管理，强化学习训练中的参数同步将自然受益于这一共享基础设施；(5) 支持更多模型形式，并持续扩展框架的模型覆盖范围。

参考文献

- Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., and Ramjee, R. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve, 2024.
- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints, 2023.
- Alizadeh, K., Mirzadeh, I., Belenko, D., Khatami-fard, K., Cho, M., Del Mundo, C. C., Rastegari, M., and Farajtabar, M. LLM in a flash: Efficient large language model inference with limited memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Anyscale. How continuous batching enables 23x throughput in LLM inference while reducing p50 latency. <https://www.anyscale.com/blog/continuous-batching-llm-inference>, 2023.
- Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The long-document transformer, 2020.
- Black Forest Labs. FLUX.1: A suite of text-to-image generation models. <https://github.com/black-forest-labs/flux>, 2024.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning*, 2024.
- Chase, H. LangChain: Building applications with LLMs through composability. <https://github.com/langchain-ai/langchain>, 2022.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling, 2023a.
- Chen, Z., Wu, J., Wang, W., Su, W., Chen, G., Xing, S., Zhong, M., Zhang, Q., Zhu, X., Lu, L., et al. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks, 2023b.
- Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning, 2023.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- Deng, C., Zhu, D., Li, K., Gou, C., Li, F., Wang, Z., Zhong, S., Yu, W., Nie, X., Song, Z., Shi, G., and Fan, H. Emerging properties in unified multimodal pretraining, 2025.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani,

- M., Minderer, M., Heigold, G., Gelly, S., et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- Du, Z., Chen, Q., Zhang, S., Hu, K., Lu, H., Yang, Y., Hu, H., Zheng, S., Guo, Y., Wang, Z., et al. CosyVoice: A scalable multilingual zero-shot text-to-speech synthesizer based on supervised semantic tokens, 2024.
- Einziger, G. and Friedman, R. TinyLFU: A highly efficient cache admission policy. In *2014 22nd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*, pp. 146–153. IEEE, 2014.
- Emu3 Team, BAAI. Emu3: Next-token prediction is all you need, 2024.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(1):5232–5270, 2022.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.
- Gangidi, A., Miao, R., Zheng, S., Bondu, S. J., Goes, G., Morsy, H., Puri, R., Riftadi, M., Shetty, A. J., Yang, J., Zhang, S., Fernandez, M. J., Gandham, S., and Zeng, H. RDMA over ethernet for distributed training at meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pp. 57–70. ACM, 2024.
- Gao, B., He, Z., Sharma, P., Kang, Q., Jevdjic, D., Deng, J., Yang, X., Yu, Z., and Zuo, P. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *USENIX Annual Technical Conference (ATC)*, 2024.
- Hintjens, P. *ZeroMQ: Messaging for Many Applications*. O’Reilly Media, 2013.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pp. 6840–6851, 2020.
- Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, M. X., Chen, D., Lee, H., Ngiam, J., Le, Q. V., Wu, Y., et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., et al. Mixtral of experts, 2024.
- Kimi Team. Kimi k1.5: Scaling reinforcement learning with LLMs, 2025.
- Kingma, D. P. and Welling, M. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pp. 611–626. ACM, 2023.
- LangChain Inc. LangGraph: Build resilient language agents as graphs. <https://github.com/langchain-ai/langgraph>, 2024.
- Lee, W., Lee, J., Seo, J., and Sim, J. InfiniGen: Efficient generative inference of large language

- models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.
- Lin, B., Zhang, C., Peng, T., Zhao, H., Xiao, W., Sun, M., Liu, A., Zhang, Z., Li, L., Qiu, X., et al. Infinite-LLM: Efficient LLM service for long context with DistAttention and distributed KV-Cache, 2024.
- Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., and Han, S. AWQ: Activation-aware weight quantization for LLM compression and acceleration, 2023.
- Liu, H., Li, C., Wu, Q., and Lee, Y. J. Visual instruction tuning, 2023.
- Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Zhang, Q., Du, K., Yao, J., Lu, S., Ananthanarayanan, G., Maire, M., Hoffmann, H., Holtzman, A., and Jiang, J. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM, 2024.
- Meituan LongCat Team, Xiao, B., et al. LongCat-Next: Lexicalizing modalities as discrete tokens, 2026.
- Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pp. 561–577, 2018.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback, 2022.
- Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 118–132. IEEE, 2024.
- Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4195–4205, 2023.
- Podell, D., English, Z., Lacey, K., Blattmann, A., Dockhorn, T., Müller, J., Peng, J., and Rombach, R. SDXL: Improving latent diffusion models for high-resolution image synthesis, 2023.
- Qin, R., Li, Z., He, W., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: A KVCache-centric disaggregated architecture for LLM serving, 2024.
- Qwen Team. Qwen2 technical report, 2024.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pp. 8748–8763. PMLR, 2021.
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. Robust speech recognition via large-scale weak supervision, 2022.
- Redis Ltd. Redis: The open-source, in-memory data structure store. <https://redis.io>, 2009.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10684–10695, 2022.

- SGLang Team. SGLang-Omni: High-performance multi-stage pipeline framework for omni models. <https://github.com/sgl-project/sglang-omni>, 2025.
- Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-LM: Training multi-billion parameter language models using model parallelism, 2019.
- Srivatsa, V., He, Z., Abhyankar, R., Li, D., and Zhang, Y. Preble: Efficient distributed prompt scheduling for LLM serving, 2024.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. RoFormer: Enhanced transformer with rotary position embedding, 2021.
- Tencent Hunyuan Team. HunyuanImage 3.0 technical report, 2025.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models, 2023.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- vLLM Team. vLLM-Omni: Fully disaggregated serving for any-to-any multimodal models, 2026.
- Wu, C., Chen, X., Wu, Z., Ma, Y., Liu, X., Pan, Z., Liu, W., Xie, Z., Yu, X., Ruan, C., et al. Janus: Decoupling visual encoding for unified multimodal understanding and generation, 2024.
- Xie, J., Mao, W., Bai, Z., Zhang, D. J., Wang, W., Lin, K. Q., Gu, Y., Chen, Z., Yang, Z., and Shou, M. Z. Show-o: One single transformer to unify multimodal understanding and generation, 2024.
- Yang, Y., Ma, H., Yin, S., Wen, Z., Li, Y., and Chen, J. KVSharer: Efficient inference via layer-wise dissimilar KV cache sharing, 2024.
- Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. ORCA: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538, 2022.
- Zhai, X., Mustafa, B., Kolesnikov, A., and Beyer, L. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 11975–11986, 2023.
- Zheng, L., Li, Z., Zhang, H., Zhuang, Y., Chen, Z., Huang, Y., Wang, Y., Xu, Y., Zhuo, D., Xing, E. P., et al. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 559–578, 2022.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs, 2024.
- Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024.

附录

A 钻石与序列依赖的嵌套情景

前文分别介绍了三种依赖类型的机制。然而，在现实世界的业务图中，多种依赖类型常常同时出现（如图 7 所示）：以 HunyuanImage-3 的图像生成流水线为例，分叉源 `parse_request` 按模态产生多个帧（多帧），每个帧经过不同的编码器后汇聚于同一汇合点（钻石依赖），而整个链路又嵌套在 `SessionLoadBalance` 的保持顺序的连接中（序列依赖）。本节展示了 `activation_id` 机制在重叠场景下依然有效，无需对每种组合进行特殊处理——首先补充泛化所需的通用公式和每帧的元数据，然后通过三个逐步递进的案例，展示钻石依赖与序列依赖重叠时的行为。

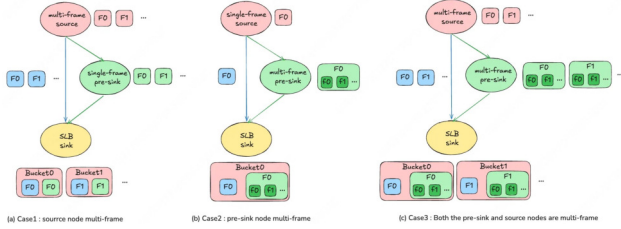


Figure 7. 钻石与序列依赖的嵌套情景

aid 的通用计算：按边二元决策。 aid 按每条出边进行计算：每当一个结点生成一个帧并将其沿一条出边发送给下游 (v, g) 时，框架会基于下游是否存在桶做出二元决策——存在桶的下游遵循规则 C，不存在桶的下游遵循规则 A。该准则针对下游结点进行判断，且按边确定；因此，同一结点的不同出边可能遵循不同的规则。

Rule C 的两个分支决定了当菱形结构与多个帧叠加时，是否产生累积或分组：当 s 命中时，种子从上游更远的分支源获取输出序列号，且该序列号在当前结点的多个帧中保持不变 $\rightarrow$ 多个帧共享相同的 aid $\rightarrow$ 累积；当 s 未命中（当前结点本身为分支源）时，种子采用当前帧的 `yidx` $\rightarrow$ 多个帧具有不同的 aid $\rightarrow$ 分组。

Algorithm 2 Compute aid for an outgoing message to downstream (v, g)

Input: current activation a , yield index $yidx$, downstream node v , OR-group g

if HASBUCKET(v, g) **and** $s \leftarrow \text{diamond_roots.lookup}(\text{ROOT}(v, g))$ hits **then**

{Rule C: use diamond-root's fixed seed $\rightarrow$ same-source frames share one bucket}

$aid \leftarrow \text{HASH}(gc, s.aid, s.node, s.yidx, g, v)$

else

{Rule A: use current node as seed $\rightarrow$ each frame gets its own bucket}

$aid \leftarrow \text{HASH}(gc, a.aid, cur, yidx, g, v)$

end if

return aid

生成所需的钻石根身份标识通过消息沿 `diamond_roots` 传播（一个钻石根状态表，`List[DiamondRootEntry]`，每个条目包含源结点、源 aid 和源产量索引）：无桶结点将传入的状态表原样传递，而钻石根结点在传出时会追加自己的条目（按源结点去重）；规则 C 随后从此表中根据根结点查找 s 。

情形 1：钻石根多帧，中间结点多帧—收敛点根据钻石根的帧进行分桶。每次钻石根产生一个帧时，其产出索引均不同；规则 C 计算出 K 个不同的辅助项，导致收敛点分裂为 K 个桶，每个桶对应钻石根的一个产出，独立地收集输入并触发。当收敛点为 SLB 结点时，还必须保持顺序：钻石根的每个产出恰好对应一个序号，且 SLB 按预期顺序释放 K 个桶，维持顺序性。

情形 2：钻石根单帧，中间结点多帧—多个帧按顺序累积于同一桶中。中间结点的钻石根位于更上游，仅产生一帧，因此其产出索引固定。中间结点各 M 产出所计算的辅助值完全相同，所有 M 帧均落入同一桶内进行累积；同时，中间结点的多帧也必须在此桶内按顺序累积。

情形 3：钻石根多帧，中间结点多帧—两层叠加。

外层根据情形 1 中钻石根的帧数被分割成多个桶；在每个桶内，对应的中间结点的多帧按情形 2 累积。两层均递归地由同一公式推导得出：钻石根的产出索引决定某帧落入哪个外层桶，而中间结点的多帧共享相同的钻石根产出索引，因此在桶内按顺序累积。

B LangGraph / vllm-omni / sglang-omni 的工作流定义示例

LangGraph

```
# Nodes = Python functions,
# Edges = add_edge / add_conditional_edges,
# State = TypedDict
import operator
from typing import TypedDict, Annotated, Any
from langgraph.graph import StateGraph, START,
    END

class S(TypedDict):
    prompt: str; image: Any; gen_audio: bool
    messages: Annotated[list, operator.add]
    streamed: Annotated[list, operator.add]
    audio: bytes

def preprocess(s): return s
def img_enc(s):
    return {"img_emb": f"vit({s.get('image','')})"}
def embed(s):
    return {"txt_emb": f"emb({s['prompt'][:10]})"}
def thinker(s):
    return {
        "messages": [{"role": "assistant",
            "content": f"ans:{s['prompt'][:10]}"}],
        "streamed": ["tok1", "tok2", "tok3"]
    }
def talker(s):
    if not s.get("gen_audio"): return {}
    return {"audio":
        f"tts({s['messages'][-1]['content'][:10]})"
        .encode()}
def tools(s):
    return {"messages":
```

```
[{"role": "tool", "content": "result"}]}}
```

```
g = StateGraph(S)
g.add_node("pre", preprocess)
g.add_node("img", img_enc)
g.add_node("emb", embed)
g.add_node("think", thinker)
g.add_node("tts", talker)
g.add_node("tools", tools)

g.add_edge(START, "pre")
g.add_edge("pre", "emb")
# Dynamic branching
g.add_conditional_edges("pre",
    lambda s: ["img", "emb"]
        if s.get("image") else ["emb"],)
g.add_edge("img", "think")
g.add_edge("emb", "think")
g.add_edge("think", "tools")
g.add_conditional_edges("tools",
    lambda s: "think"
        if s["messages"][-1].get("tool_calls")
        else END,
    {"think": "think", END: END})
g.add_conditional_edges("think",
    lambda s: ["tts"]
        if s.get("gen_audio") else [END],)
g.add_edge("tts", END)
graph = g.compile()
```

```
async for e in graph.astream_events(
    INPUT, version="v2"):
    if (e["event"] == "on_chain_end"
        and "streamed" in
            e["data"].get("output", {})):
        print(e["data"]["output"]["streamed"])
```

vllm-omni

```
# Nodes = frozen dataclass,
# Edges = input_sources tuple,
# Bridge = function reference

P = PipelineConfig(model_type="m", stages=(
    StagePipelineConfig(
        stage_id=0, model_stage="thinker",
        execution_type=
            StageExecutionType.LLM_AR,
        input_sources=(),
        requires_multimodal_data=True,
```

```

        engine_output_type="text",
    ),
    StagePipelineConfig(
        stage_id=1, model_stage="talker",
        execution_type=
            StageExecutionType.LLM_GENERATION,
        input_sources=(0,),
        engine_output_type="audio",
        async_chunk_process_next_stage_input_func
    =
        "m.thinker2talker_chunk",
        sync_process_input_func=
        "m.thinker2talker_sync",
    ),
))

def thinker2talker_sync(src, **kw):
    return [OmniTokensPrompt(
        prompt_token_ids=[0],
        additional_information={
            "text": src[0].outputs[0].text})]

```

sglang-omni

```

# Nodes = JSON StageConfig + factory functions,
# Edges = next/route_fn/wait_for/stream_to

# pipeline.json
{
    "stages": [
        {"name": "pre", "factory": "s.create_pre",
         "route_fn": "n.pre_next"},
        {"name": "img", "factory": "s.create_img",
         "next": "agg"},
        {"name": "agg", "factory": "s.create_agg",
         "wait_for": ["pre", "img"], "next": "think"},
        {"name": "think", "factory": "s.create_think",
         "stream_to": ["tts", "dec"]},
        {"name": "tts", "factory": "s.create_tts",
         "terminal": true},
        {"name": "dec", "factory": "s.create_dec",
         "terminal": true}
    ]
}

# next_stage.py
def pre_next(rid, out):
    s = load_state(out)
    st = ["agg"]
    if s.has_image(): st.insert(0, "img")

```

```

    if s.has_audio(): st.insert(0, "aud")
    return st

```

```

# stages.py
def create_think(model_path):
    eng = create_sglang_ar_engine(...)
    def _stream(payload, token):
        return {"token_id": int(token)}
    return EngineExecutor(eng,
        stream_builder=_stream)

```

C 使用人工智能编程

Omni-Flow 于 2025 年末设计完成，开发工作始于 2026 年初，工程投入总计约 1-2 人·半年，其中绝大多数工作是在人工智能协助下完成的。迭代过程大致分为三个阶段：(i) 人类设计架构，AI 处理局部填充——早期的人工智能尚不具备全局设计能力，因此由架构师主导结构设计，而人工智能负责填充方法实现和样板代码；(ii) AI 修复缺陷，人类监督——框架成型后，大量缺陷修复和边缘情况处理交由人工智能完成，人类负责审查正确性和一致性；(iii) 人类提供设计文档，AI 重构框架——随着人工智能推理能力提升以及设计趋于收敛，人工智能逐步接管重构工作，包括根据详细设计文档对控制流 v1→v2 进行完全重写。

我们遇到了几个反复出现的挑战，并由此塑造了我们的实践方法：(1) AI 倾向于局部修补，不断积累框架层面的临时解决方案——我们要求每个修复都必须评估是否解决了根本原因，对仅是位置迁移的修改予以回滚；(2) AI 生成代码的可读性持续下降——我们要求持续进行人工监督，逐项阅读每个完成的模块，以确认其与整体设计的一致性；(3) 在并行提交 AI 代码时的正确性控制——AI 生成代码的速度过快，无法通过人工对比差异进行审查，因此每个模块都必须配有单元测试，所有新增逻辑必须附带相应的测试，使测试覆盖成为唯一可靠的保障；(4) AI 在大型模块上容易发散——我们首先将需求收敛为文档，明确接口定义、核心逻辑和边界条件，然后要求 AI 严格依据该文档执行。